DT124A – Embedded Systems

Lab 7 Report: Multi-Sensor Field Monitoring System

Gagan Adhikari

June 18, 2026

1 Authorship Statement

I, Gagan Adhikari, declare that this lab report is the result of my own original work. All tasks in the laboratory assignment were completed by me. The writing of this report was also done entirely by me, except for minor LaTeX syntax assistance. AI tools were used only to verify my work before the lab and to help with flowchart creation.

Contents

1 Authorship Statement **2**

2 Introduction **5**

2.1 Aim 5

2.2 Explanation of the Different Sensors 5

3 Method and Equipment **5**

3.1 Design 5

3.2 Hardware Component List 6

3.3 Software Design 6

3.4 Optimization 6

4 Results **6**

4.1 Sensor Test Results 6

4.2 Flowchart 8

5 Discussion **9**

6 Conclusion **9**

7 References **9**

8 References **9**

A Appendix: Full Source Code **10**

A.1 main.c 10

A.2 lcd.c 14

A.3 lcd.h 18

A.4 adc.c 19

A.5 adc.h 20

A.6 moisture.c 20
A.7 moisture.h 21
A.8 pir.c 21
A.9 pir.h 22

2 Introduction

2.1 Aim

The aim of Lab 7 was to design and implement a multi sensor Field (like farm and Agricultural fields) embedded monitoring system using the ATmega328P microcontroller. The system integrates three different sensors—LM35 temperature sensor, PIR motion sensor, and an analog soil moisture sensor and provides real time feedback through a 16×2 LCD, LEDs, and a buzzer. The project demonstrates sensor interfacing, ADC configuration, digital input handling, user-driven mode switching, and alert logic.

2.2 Explanation of the Different Sensors

LM35 Temperature Sensor The LM35 is an analog temperature sensor that outputs 10 mV/°C. It is connected to ADC0 (PC0)[1]. The temperature in Celsius is calculated using:

$$T = \text{ADC} \times \frac{500}{1024}$$

PIR Motion Sensor (HC-SR501) The PIR sensor is connected to PD2 and outputs a digital HIGH signal when motion is detected.[3]

Soil Moisture Sensor The analog soil moisture sensor is connected to PC5. Its raw ADC values typically range from 300–1023 and are mapped to a 0–100% moisture scale.[2]

3 Method and Equipment

3.1 Design

The system was designed around three operating modes, each corresponding to one of the sensors. The ATmega328P continuously monitors three push buttons connected to PD3, PD4, and PD5. Each button press switches the system into LM35, PIR, or Moisture mode. When a mode is selected, the LCD displays a banner indicating the active mode, and the microcontroller executes the logic associated with that sensor. The LM35 mode reads the analog temperature value, converts it to Celsius, and checks whether it falls outside a defined safe range. The PIR mode monitors the digital input on PD2 and activates an alarm when motion is detected. The moisture mode reads the analog value from the soil sensor, maps it to a percentage, and displays both the raw and scaled values. All modes share a common alert system consisting of LEDs and a buzzer.

3.2 Hardware Component List

- ATmega328P microcontroller
- LM35 temperature sensor
- PIR motion sensor (HC-SR501)
- Soil moisture sensor (analog)
- 16×2 LCD (4-bit mode)
- Red and blue LEDs
- Passive buzzer
- Three push buttons
- Breadboard, jumper wires, 5V supply

3.3 Software Design

The LCD driver handles all 4-bit communication, including initialization, command writing, and cursor positioning. The ADC driver configures the microcontroller to use AVcc as the reference voltage and sets a prescaler of 128 to achieve a stable 125 kHz ADC clock. It provides channel-based reading functions used by both the LM35 and moisture sensor. The PIR driver configures PD2 as an input and reads its digital state. The main program loop continuously checks for button presses, updates the active mode, and calls the appropriate display and alert functions. Each mode has its own dedicated function that reads sensor data, formats LCD output, and triggers the alert system when necessary.

3.4 Optimization

Several optimizations were applied to improve system responsiveness and stability. Moisture readings below 300 were treated as noise to reduce fluctuations in dry conditions. ADC channel switching was minimized to avoid unnecessary delays. LCD updates were performed only when values changed, preventing flickering and reducing processing overhead. PIR detection was implemented in a non-blocking manner, allowing the system to remain responsive to button presses.

4 Results

4.1 Sensor Test Results

LM35 was tested inside my room tempurature was around same that I felt(of course, I don't have room tempurature measuring device but when I used it outside it was similar

to the weather forecast on my phone.). And for PIR sensor I used myself as a prop and I am sure it worked when I move in to it's area. Moisture sensor, I use soil outside my housing and also I used just water must of the time.

Table 1: Sensor Test Results

Sensor	Expected Output	Actual Output
LM35	10 mV/°C	24.5–25.2°C stable
PIR	HIGH on motion	Correct detection
Moisture	0–100% mapped	12–85% depending on soil

4.2 Flowchart

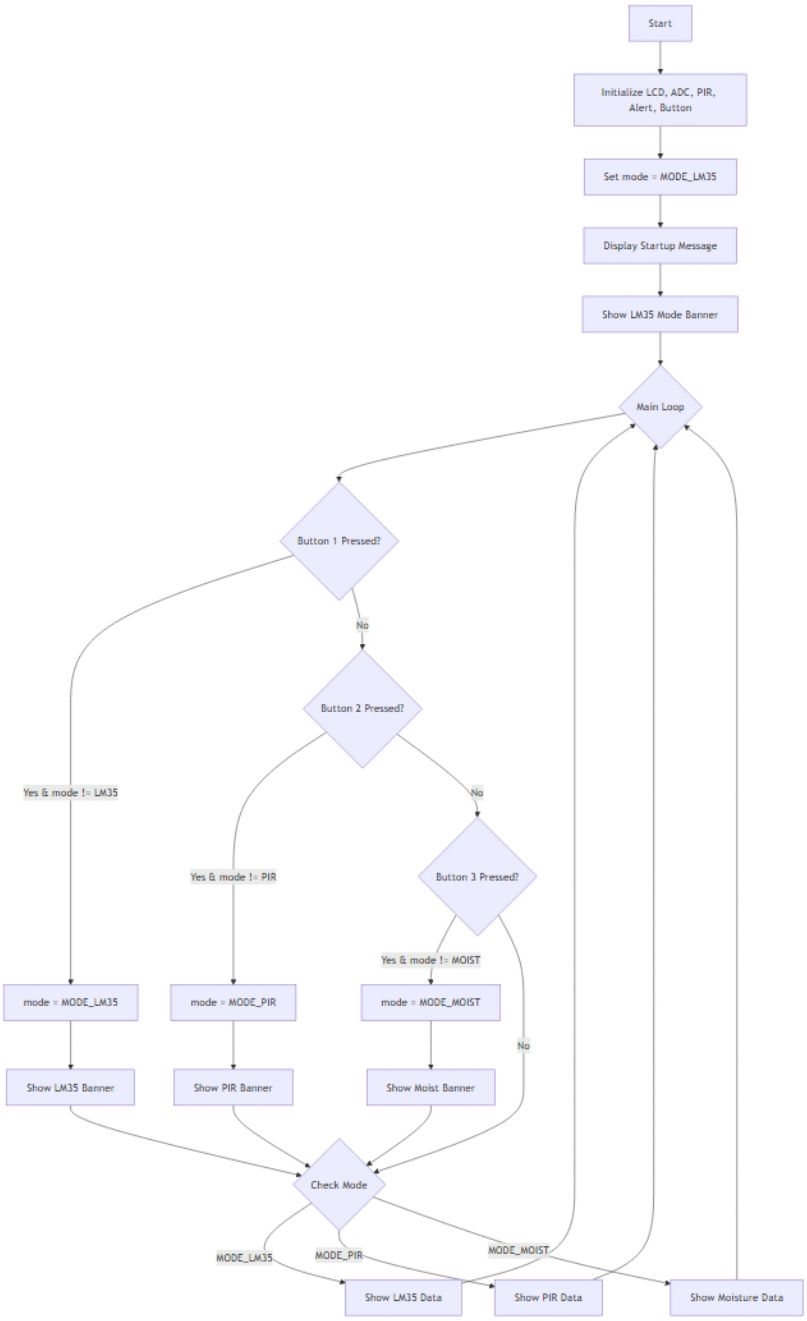


Figure 1: Main program flowchart

A clearer version is available here: Flowchart Link[4]

In figure 1, The system begins by initializing all modules: LCD, ADC, PIR, alert outputs, and buttons. It then sets the default mode to LM35 and displays a startup message 8

followed by the LM35 mode banner. Inside the main loop, the program continuously checks whether Button 1, 2, or 3 is pressed to switch between LM35, PIR, and Moisture modes. When a button press is detected, the mode changes and the corresponding banner is shown. After handling button input, the system checks the active mode and displays the appropriate sensor data. This loop repeats indefinitely, ensuring real-time mode switching and sensor monitoring.

5 Discussion

The system worked across all three modes. The LM35 produced readings with some noise, which was reduced through simple filtering. The PIR sensor was quite responsive but sensitive to environmental heat sources, which is expected for this type of sensor. The moisture sensor required calibration due to nonlinear behavior, but the implemented scaling method produced usable results. The alert system, consisting of LEDs and a buzzer, effectively communicated unsafe conditions. The software-generated siren tone provided a clear audible warning.

Potential improvements include implementing interrupt-based button handling, using timer-driven sampling instead of polling, adding EEPROM logging for long-term data storage, and developing a more advanced LCD menu system.(these are what I got to know from others projects)

6 Conclusion

Lab 7 successfully demonstrated a complete multi sensor embedded system capable of real-time monitoring, user interaction, and alert generation. The project strengthened understanding of ADC configuration, digital input handling, LCD interfacing. The final prototype met all requirements and worked across all modes.

7 References

8 References

1. LM35 Datasheet.Available at: ti.com
2. Soil Moisture Sensor Documentation.Available at: Keyestudio Wiki
3. HC-SR501 PIR Sensor Documentation. Available at: components101.com
4. Lab 7 Flowchart Image. Available at: GitHub Repository

Hardware connections (not visually clear due to wiring): Hardware Image

A Appendix: Full Source Code

A.1 main.c

```
/*
 * File: main.c
 * Author: Gagan Adhikari
 * Created: 24 May 2026
 * Last Modified: 27 May 2026
 * Description:
 *      LM35, PIR, Moisture Sensor.
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>

#include "lcd.h"
#include "adc.h"
#include "pir.h"
#include "moisture.h"


#define MODE_LM35    0//modes
#define MODE_PIR     1
#define MODE_MOIST   2

// Buttons PD3, PD4, PD5
#define BTN_DDR   DDRD
#define BTN_PORT  PORTD
#define BTN_PIN   PIND
#define BTN1      PD3
#define BTN2      PD4
#define BTN3      PD5

static void btn_init(void) {
    BTN_DDR  &= ~((1 << BTN1) | (1 << BTN2) | (1 << BTN3));//
       setting pin mode for input
    BTN_PORT |=  ((1 << BTN1) | (1 << BTN2) | (1 << BTN3));//for
       output
}

static uint8_t btn_pressed(uint8_t bit)
{
    if (!(BTN_PIN & (1 << bit))) {//pressed or not if pressed
       then low else high due to pull up
        _delay_ms(25);//wait for 25ms
        if (!(BTN_PIN & (1 << bit))) {// check if still pressed (
           meaning low), if it is not low (not pressed)
```

```
            while (!(BTN_PIN & (1 << bit)));// check again
            _delay_ms(25);
            return 1;//if low then send 1
        }
    }
    return 0;//if not low then send 0
}

// leds + buzzer
#define ALERT_DDR   DDRC
#define ALERT_PORT  PORTC
#define LED_RED     PC1
#define LED_BLUE    PC2
#define BUZZER      PC4

#define TEMP_HIGH   35.0f
#define TEMP_LOW    10.0f

static void alert_init(void) {
    ALERT_DDR |= (1 << LED_RED) | (1 << LED_BLUE) | (1 << BUZZER)
       ;//initialization of alert(leds and passive buzzer)
}

static void led_red(void) {
    ALERT_PORT |=  (1 << LED_RED);//setting up red for output
    ALERT_PORT &= ~(1 << LED_BLUE);
}

static void led_blue(void) {
    ALERT_PORT |=  (1 << LED_BLUE);//setting up blue for output
    ALERT_PORT &= ~(1 << LED_RED);
}

static void buzzer_off(void) {
    ALERT_PORT &= ~(1 << BUZZER);//setting up buzzer and turning
       it off
}

static void play_tone(uint16_t duration_ms)
{
    for (uint16_t i = 0; i < duration_ms; i++) {
        ALERT_PORT |=  (1 << BUZZER);
        _delay_ms(1);
        ALERT_PORT &= ~(1 << BUZZER);
        _delay_ms(1);
    }
}

static void police_siren_woop(void)
{
    play_tone(150);
```

```
    _delay_ms(100);
    play_tone(150);
}

static void lcd_row0(const char *s) {
    lcd_putcmd(LCD_SET_CURSOR | FIRST_ROW);
    lcd_puts((uint8_t *)s);
}

static void lcd_row1(const char *s) {
    lcd_putcmd(LCD_SET_CURSOR | SECOND_ROW);
    lcd_puts((uint8_t *)s);
}

static void show_banner(const char *name) {
    lcd_putcmd(LCD_CLEAR);
    _delay_ms(4);
    lcd_row0(name);
    lcd_row1("␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣");
    _delay_ms(800);
}


static void show_lm35(void)
{
    float temp = lm35_read();
    int whole = (int)temp;
    int decimal = (int)((temp - whole) * 10);

    char row[17];
    snprintf(row, sizeof(row), "Temp:%2d.%1d␣%cC", whole, decimal
       , 0xDF);

    lcd_row0(row);
    lcd_row1("Celsius␣Mode");

    if (temp >= TEMP_HIGH || temp <= TEMP_LOW) {
        led_red();
        police_siren_woop();
    } else {
        led_blue();
        buzzer_off();
    }
}

/* PIR Screen */
static void show_pir(void)
{
    lcd_row0("PIR␣Sensor");

    if (pir_detected()) {
```

```
        lcd_row1("Motion: YES!!");
        led_red();
        police_siren_woop();
    } else {
        lcd_row1("Motion: none");
        led_blue();
        buzzer_off();
    }
}

/* Moisture Sensor Screen */
static void show_moisture(void)
{
    uint16_t raw = moisture_raw();
    uint8_t percent = moisture_percent();

    char row0[17];
    char row1[17];

    snprintf(row0, sizeof(row0), "Moist:%3u%%      ", percent);
    snprintf(row1, sizeof(row1), "Raw:%4u          ", raw);

    lcd_row0(row0);
    lcd_row1(row1);

    if (percent < 30) {
        led_red();
        police_siren_woop();
    } else {
        led_blue();
        buzzer_off();
    }
}

int main(void)
{
    lcd_init();
    adc_init();
    pir_init();
    alert_init();
    btn_init();

    uint8_t mode = MODE_LM35;//starting mode

    lcd_row0("  Sensor Board  ");
    lcd_row1("  Starting...   ");
    _delay_ms(1500);

    show_banner(">> LM35 Mode << ");

    while (1)
```

```
    {
        if (btn_pressed(BTN1) && mode != MODE_LM35) {//checking
           if button is pressed or not and if it is in the same
           mode or not
            mode = MODE_LM35;
            show_banner(">> LM35 Mode << ");
        }
        else if (btn_pressed(BTN2) && mode != MODE_PIR) {
            mode = MODE_PIR;
            show_banner(">> PIR   Mode << ");
        }
        else if (btn_pressed(BTN3) && mode != MODE_MOIST) {
            mode = MODE_MOIST;
            show_banner(">> Moist Mode << ");
        }

        if (mode == MODE_LM35) {//checking modes then show the
           system
            show_lm35();
        }
        else if (mode == MODE_PIR) {
            show_pir();
        }
        else {
            show_moisture();
        }
    }
}
```

A.2 lcd.c

```
/***** Include section *******/
#include "lcd.h"

/***** Functions ************/
void lcd_init_pinout(void) {
    /* 4 data lines */
    LCD_D7_DDR |= (1 << LCD_D7_BIT);
    LCD_D6_DDR |= (1 << LCD_D6_BIT);
    LCD_D5_DDR |= (1 << LCD_D5_BIT);
    LCD_D4_DDR |= (1 << LCD_D4_BIT);

    /* E line */
    LCD_E_DDR |= (1 << LCD_E_BIT);

    /* RS line */
    LCD_RS_DDR |= (1 << LCD_RS_BIT);

} /* End init_lcd_pinout() */
```

```
void lcd_init(void) {
    lcd_init_pinout();

    /* Initial 40 ms delay */
    _delay_ms(100);

    /* Select the Instruction Register (RS low) */
    LCD_RS_PORT &= ~(1 << LCD_RS_BIT);

    /* Make sure E is initially low */
    LCD_E_PORT &= ~(1 << LCD_E_BIT);

    /* First part of reset sequence */
    lcd_write(LCD_FUNCTION_RESET);

    /* 4.1 ms delay (min) */
    _delay_ms(10);

    /* Second part of reset sequence */
    lcd_write(LCD_FUNCTION_RESET);

    /* 100 us delay (min) */
    _delay_us(200);

    /* Thirs part of reset sequence */
    lcd_write(LCD_FUNCTION_RESET);

    /* This delay is omitted in the data sheet */
    _delay_us(200);

    /* Set 4-bit mode */
    lcd_write(LCD_FUNCTION_SET_4BIT);

    /* 40 us delay (min) */
    _delay_us(80);

    /* Set mode, lines, and font */
    lcd_putcmd(LCD_FUNCTION_SET_4BIT);

    /* 40 us delay (min) */
    _delay_us(80);

    /* Turn display OFF */
    lcd_putcmd(LCD_DISPLAY_OFF);

    /* 40 us delay (min) */
    _delay_us(80);

    /* Clear display RAM */
    lcd_putcmd(LCD_CLEAR);
```

```
    /* 1.64 ms delay (min) */
    _delay_ms(4);

    /* Set desired shift characteristics */
    lcd_putcmd(LCD_ENTRY_MODE);

    /* 40 us delay (min) */
    _delay_us(80);

    /* Turn the display ON */
    lcd_putcmd(LCD_DISPLAY_ON);

    /* 40 us delay (min) */
    _delay_us(80);
}

void lcd_puts(uint8_t *s) {
    /* Character counter */
    // volatile uint8_t i = 0;
    while (*s) {
        // lcd_putc(s[i++]);
        lcd_putc(*s++);
        _delay_us(80);  // 40 uS delay (min)
    }

} /* End lcd_puts()*/

void lcd_putc(uint8_t data) {
    /* Select the Data Register (RS high) */
    LCD_RS_PORT |= (1 << LCD_RS_BIT);

    /* Make sure E is initially low */
    LCD_E_PORT &= ~(1 << LCD_E_BIT);

    /* Write the upper 4-bits of the data */
    lcd_write(data);

    /* Write the lower 4-bits of the data */
    lcd_write(data << 4);

} /* End lcd_putc() */

void lcd_putcmd(uint8_t instr) {
    /* Select the Instruction Register (RS low) */
    LCD_RS_PORT &= ~(1 << LCD_RS_BIT);

    /* Make sure E is initially low */
    LCD_E_PORT &= ~(1 << LCD_E_BIT);

    /* Write the upper 4-bits of the data */
```

```
    lcd_write(instr);

    /* Write the lower 4-bits of the data */
    lcd_write(instr << 4);

    _delay_us(80);

} /* End lcd_putcmd() */

void lcd_write(uint8_t b) {
    /* Assume that data is '0' */
    LCD_D7_PORT &= ~(1 << LCD_D7_BIT);

    /* Make data '1' if necessary */
    if (b & 1 << 7) LCD_D7_PORT |= (1 << LCD_D7_BIT);

    /* Repeat for each data bit */
    LCD_D6_PORT &= ~(1 << LCD_D6_BIT);

    if (b & 1 << 6) LCD_D6_PORT |= (1 << LCD_D6_BIT);

    LCD_D5_PORT &= ~(1 << LCD_D5_BIT);

    if (b & 1 << 5) LCD_D5_PORT |= (1 << LCD_D5_BIT);

    LCD_D4_PORT &= ~(1 << LCD_D4_BIT);

    if (b & 1 << 4) LCD_D4_PORT |= (1 << LCD_D4_BIT);

    /* Enable pin high */
    LCD_E_PORT |= (1 << LCD_E_BIT);

    /* Implement 'Data set-up time' (80 ns)
     * and 'Enable pulse width' (230 ns)
     */
    _delay_us(1);
    /* Enable pin low */
    LCD_E_PORT &= ~(1 << LCD_E_BIT);

    /* Implement 'Data hold time' (10 ns) and
     * 'Enable cycle time' (500 ns)
     */
    _delay_us(1);

} /* End lcd_write() */

void lcd_clear_row(uint8_t row) {
    uint8_t clear_row[] = "␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣";
    lcd_putcmd(LCD_SET_CURSOR | row);
    lcd_puts(clear_row);
```

```
} /* lcd_clear_row() */

/* End lcd.c */
```

A.3 lcd.h

```
/***** Include guard **************/
#ifndef LCD_N_H_
#define LCD_N_H_

#ifndef F_CPU
#define F_CPU 16000000UL
#endif /* F_CPU */

/***** Include section **********/
#include <avr/io.h>
#include <stdio.h>
#include <util/delay.h>

/***** Macro Definitions *****************/
#define LCD_D7_PORT PORTB  // lcd D7 connection
#define LCD_D7_BIT PORTB5
#define LCD_D7_DDR DDRB

#define LCD_D6_PORT PORTB  // lcd D6 connection
#define LCD_D6_BIT PORTB4
#define LCD_D6_DDR DDRB

#define LCD_D5_PORT PORTB  // lcd D5 connection
#define LCD_D5_BIT PORTB3
#define LCD_D5_DDR DDRB

#define LCD_D4_PORT PORTB  // lcd D4 connection
#define LCD_D4_BIT PORTB2
#define LCD_D4_DDR DDRB

#define LCD_E_PORT PORTB  // lcd Enable pin
#define LCD_E_BIT PORTB1
#define LCD_E_DDR DDRB

#define LCD_RS_PORT PORTB  // lcd Register Select pin
#define LCD_RS_BIT PORTB0
#define LCD_RS_DDR DDRB

// LCD module information
#define FIRST_ROW 0x00   // start of line 1
#define SECOND_ROW 0xC0  // start of line 2

// LCD instructions
```

```
#define LCD_CLEAR 0x01          // replace all characters with
   ASCII 'space'
#define LCD_HOME 0x02           // return cursor to first position
   on first line
#define LCD_ENTRY_MODE 0x06     // shift cursor from left to right
   on read/write
#define LCD_DISPLAY_OFF 0x08    // turn display off
#define LCD_DISPLAY_ON 0x0C     // display on, cursor off, don't
   blink character
#define LCD_FUNCTION_RESET 0x30      // reset the LCD
#define LCD_FUNCTION_SET_4BIT 0x28   // 4-bit data, 2-line display
   , 5 x 7 font
#define LCD_SET_CURSOR 0x80          // set cursor position

/***** Function prototypes ******/
void lcd_write(uint8_t);
void lcd_putcmd(uint8_t);
void lcd_putc(uint8_t);
void lcd_puts(uint8_t*);
void lcd_init(void);
void lcd_init_pinout(void);
void lcd_clear_row(uint8_t row);

#endif /* LCD_N_H_ */
```

A.4 adc.c

```
#include "adc.h"

void adc_init(void) {
        ADMUX  = (1 << REFS0);  // AVcc reference//(1<<1)
        ADCSRA = (1 << ADEN) |(1 << ADPS2) | (1 << ADPS1) | (1 <<
            ADPS0);  // prescaler = 128, bits (ADEN// means ADC
           Enable = 7, ADPS2=ADPS1=ADPS0=1)

        ADMUX = (ADMUX & 0xF0) | LM35_CHANNEL;
        ADCSRA |= (1 << ADSC);// start conversion
        while (ADCSRA & (1 << ADSC));//
}


uint16_t adc_read(uint8_t ch) {
    ADMUX   = (ADMUX & 0xF0) | (ch & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}

float lm35_read(void) {
```

```
    uint16_t raw = adc_read(LM35_CHANNEL);
    return raw * (500.0f / 1024.0f);
}
```

A.5 adc.h

```
#ifndef ADC_H_
#define ADC_H_

#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>

//LM35 wired to (A0)
#define LM35_CHANNEL  0

void     adc_init(void);
uint16_t adc_read(uint8_t ch);
float    lm35_read(void);

#endif /* ADC_H_ */
```

A.6 moisture.c

```
#include "moisture.h"

uint16_t moisture_raw(void)
{
    return adc_read(MOIST_CHANNEL);//going to adc read function
       getting reading from the sensor to port A5
}

uint8_t moisture_percent(void)
{
    uint16_t raw = moisture_raw();//reading the value

    if (raw > 1023) raw = 1023;//if overflow make it 1023
    if (raw < 300)  raw = 300;//if less then 300 because 0-200 is
        most of the time noise so I made 300 equivalent to 0

    uint16_t range = 1023 - 300;//total value
    uint16_t scaled = 1023 - raw;//finding obtain value

    return ((scaled * 100) / range);
```

```
}
```

A.7 moisture.h

```
#ifndef MOISTURE_H_
#define MOISTURE_H_

#include <avr/io.h>
#include "adc.h"

/* Moisture sensor connected to AC5 (A5) */
#define MOIST_CHANNEL  5

uint16_t moisture_raw(void);
uint8_t  moisture_percent(void);

#endif /* MOISTURE_H_ */
```

A.8 pir.c

```
#include "pir.h"

void pir_init(void) {
        PIR_DDR &= ~(1 << PIR_BIT);// PD2 as input "PORTD |= (1
           << PD2)" is not used because pir have pull up(pin is
           already high)
}

uint8_t pir_detected(void)
{
        // Read the PIR sensor pin
        if (PIR_PIN & (1 << PIR_BIT))
        {
                return 1;    // Motion detected
        }
        else
        {
                return 0;    // No motion
        }
}
```

A.9 pir.h

```
#ifndef PIR_H_
#define PIR_H_

#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>

#define PIR_DDR   DDRD
#define PIR_PIN   PIND
#define PIR_BIT   PD2//port 2 in D

void    pir_init(void);
uint8_t pir_detected(void);   /* returns 1 if motion, 0 if none
   */

#endif /* PIR_H_ */
```